

Token Importance Unit

Software & Reference-Model Overview

Date: 2026-07-18
Status: Block 3 of 4 — RTL signed off, reference model at parity
Audience: Compiler engineers integrating against the chip
Repository: LonghornSilicon/token-importance-unit

Purpose. This document is the orientation map for a compiler team integrating against the Token Importance Unit. It points to the bit-accurate reference model, the ISA spec, the RTL and its test vectors, and the compiler entry point. Read this first; then jump to the specific references it links.

Contents

1	What the TIU Is	3
2	The Software Stack	3
3	Where Things Live	3
4	Build and Test (One Line Each)	4
5	The Compiler Entry Point	4
6	Recommended Reading Order	4
7	Integration Phases	5
8	Verification Summary	5

1. What the TIU Is

The Token Importance Unit (TIU) is block 3 of the four-block LonghornSilicon LLM inference accelerator targeting TSMC 16nm FinFET (N16FFC). It is the H2O heavy-hitter eviction core for the KV cache: it accumulates each cached token's received attention mass, evicts the least-attended token when the cache is full, and emits a per-token keep/demote tier signal to the KV Cache Engine (block 2).

The silicon core is deliberately small — *accumulate + argmin + tier*. The H2O recent-window protection and KV-budget bookkeeping are a control-layer wrapper the compiler emits around the core, exactly as block 1 shipped just the ratio gate.

2. The Software Stack

Four artifacts make up the compiler-facing software for this block:

Table 1: Software artifacts and their status.

Artifact	Role	Status
rtl/token_importance_unit.sv	The DUT (parameterized SystemVerilog)	Sky130 signed off, 0 violations
sw/reference_model/tiu_ref.py	Bit-accurate Python model	40/40 evictions vs RTL
rtl/tb/testvectors/tiu_trace.hex	Golden replay trace (real Qwen2)	48 tokens, 40 evictions
sw/reference_model/example_compiler_use.py	Compiler entry point (H2O wrapper)	Runnable

The reference model is the contract: it exposes the same functional behavior the hardware implements (same saturating accumulators, same serialized-argmin eviction, same tier comparator), so a compiler backend can target the TIU before silicon or an FPGA prototype exists.

3. Where Things Live

```

rtl/
+-- token_importance_unit.sv      (the DUT)
+-- tb/
    +-- tb_token_importance_unit.sv (directed, 29/29)
    +-- tb_realdata.sv             (real-Qwen2 replay, 40/40)
    +-- testvectors/tiu_trace.hex  (committed golden trace)

sw/reference_model/
+-- README.md                     (API reference)
+-- tiu_ref.py                    (bit-accurate model + self-test)
+-- test_tiu_ref.py               (Python<->RTL parity on the trace)
+-- example_compiler_use.py       (budget sizing -> schedule -> tiers)

analysis/
```

```

+-- gen_tiu_testvectors.py          (regenerates the golden trace)
+-- h2o_*.py, full_stack_integration.py (algorithm studies)

docs/
+-- isa/token_importance_unit_isa.pdf (ISA / interface spec)
+-- tier_handshake.md                 (TIU<->KVCE tier protocol)
+-- sky130_signoff.md                 (0-violation physical sign-off)
+-- sw_overview.pdf                   (this document)

```

4. Build and Test (One Line Each)

```

# RTL simulation (icarus verilog)
cd rtl && make sim          # directed TB:   29/29 pass
cd rtl && make sim_realdata  # replay TB:    40/40 evictions bit-exact

# Reference model (Python 3.10+, no dependencies)
python sw/reference_model/tiu_ref.py          # self-test
python sw/reference_model/test_tiu_ref.py      # Python<->RTL parity:
40/40
python sw/reference_model/example_compiler_use.py# compiler walkthrough

```

The parity test replays the committed golden trace through the Python model and checks that all 40 eviction victims match the trace’s expected column (the same victims the SystemVerilog replay TB checks against the DUT). Agreement on all 40 establishes Python↔RTL bit-exactness.

5. The Compiler Entry Point

`example_compiler_use.py` is the shape a compiler backend’s codegen should produce. It drives the reference model at three levels:

1. **Budget sizing.** Turn a context length into a cache size C and recent window L from the validated gold config (25% KV budget, 50/50 recency/heavy-hitter split), clamped to the physical `N_SLOTS`.
2. **Eviction schedule.** Stream a token’s attention over the cache, drive `LOAD/ACC/EVICT`, and collect the eviction schedule the backend emits to the KV cache engine. Heavy hitters are protected; background tokens are evicted.
3. **Keep/demote tiers.** Calibrate `tier_threshold` from accumulated mass and read per-token keep (value @ CQ-8) / demote (value @ CQ-4) tiers off the core.

6. Recommended Reading Order

1. **This document** (top to bottom; you are almost done).
2. `docs/isa/token_importance_unit_isa.pdf`, especially §4 (worked lowering example) and §5 (tier handshake).
3. `docs/tier_handshake.md` for the TIU↔KVCE protocol (keep/demote/evict semantics), verified with the ACU in the loop.

4. `sw/reference_model/example_compiler_use.py` — run it, then read it side by side. The shape of that script is the shape your codegen should produce.
5. `paper/token_importance_unit.pdf` for the algorithm validation, RTL verification, and Sky130 sign-off in full.

7. Integration Phases

The interface (ISA + reference model) is stable across all phases; only the runtime implementation of the operations changes: Python function call $\rightarrow$ AXI register write $\rightarrow$ PCIe MMIO.

Table 2: Four-phase compiler integration plan.

Phase	Timeline	Compiler targets	We provide
0	now	Python reference model	Model, ISA, parity test
1	when ZCU102/104 arrives	AXI-Lite + AXI-Stream on Zynq UltraScale+	Vivado bitstream + PYNQ driver
2	all four blocks land	Same AXI; tier handshake to KVCE on HW	Multi-block FPGA project
3	post-tape-out (2027+)	PCIe-attached custom chip	TSMC 16FFC silicon + Linux driver

8. Verification Summary

Table 3: Current verification status.

Surface	Test	Status
RTL directed	<code>tb_token_importance_unit.sv</code>	29/29 pass
RTL real-data replay	<code>tb_realdata.sv</code> (Qwen2 trace)	40/40 evictions
Python reference	self-test (directed cases)	pass
Python $\leftrightarrow$ RTL parity	golden-trace replay	40/40 evictions
Physical	Sky130 LibreLane/OpenROAD sign-off	0 violations
Three-block stack	TIU + KVCE + APA on Qwen2	$\Delta -0.031$ vs FP16

This document is the canonical entry point for a compiler engineer working with the LonghornSilicon Token Importance Unit. It is versioned alongside the reference model and updated as the block evolves.